

FishMaster AI: 스마트 낚시 가이드 플랫폼

 **프로젝트 소개** 개인 낚시꾼을 위한 AI 기반 스마트 가이드 플랫폼입니다. 이미지 인식 기술과 위치 데이터 API를 결합하여 안전하고 법규를 준수하는 낚시 환경을 제공합니다.

✂ 주요 기능

1. 이미지 기반 어종 판별

- **AI 어종 분석:** 촬영된 물고기 이미지를 분석하여 어종을 즉시 식별합니다.
- **식용 여부 확인:** 해당 어종의 식용 가능 여부 및 법적 보호종(금지어종) 여부를 즉시 안내합니다.

2. 체장 측정 및 규제 준수 가이드

- **실시간 체장 분석:** 이미지 내 물고기의 크기를 측정합니다.
- **법적 포획 기준 비교:** 분석된 체장이 해당 어종의 법적 '금지 체장' 기준을 위반하는지 실시간으로 판단하여 경고 또는 알림을 제공합니다.

3. 낚시 가능 구역 안내

- **위치 기반 필터링:** Google Maps API 및 Kakao Maps API를 연동합니다.
- **구역 정보 제공:** 사용자의 현재 위치를 기반으로 낚시 금지 구역 여부를 판별하여 정보를 제공합니다.

4. 낚시 지수 기반 AI 챗봇

- **실시간 환경 데이터 분석:** 기상 데이터, 조석 정보(물때), 풍속 등 낚시에 중요한 환경 지표를 종합 분석합니다.
- **낚시 적합도 안내:** 현재 위치와 시간에 따른 '낚시 적합 지수'를 계산하여 출조 여부를 결정하는 데 도움을 줍니다.
- **맞춤형 상담:** 사용자의 질문에 대해 현재 환경을 바탕으로 한 맞춤형 출조 조언 및 낚시 팁을 실시간으로 답변합니다.

시스템 아키텍처

이미지 기반 어종 판별

1. 데이터 구성

식용 가능 어종

- 총 **49종**
- 어종별 최소 **3장 이상의 이미지**
- 이미지 유형
 - 어류 전체 모습이 선명한 사진
 - 물속에서 촬영된 사진
 - 실제 낚시로 포획된 사진
 - 다양한 방향과 배경에서 촬영된 사진

식용 불가 어종

- 총 **3종**
 - 국매리복
 - 기름치
 - 별복
- 어종별 최소 **3장 이상의 이미지**
- 각 어종의 식용 불가 사유를 별도 JSON 데이터로 관리

어종별 3장은 시스템 동작 확인을 위한 최소 데이터입니다. 실제 AI 학습 정확도를 높이기 위해서는 어종별 최소 50장 이상, 가능하면 100장 이상의 이미지가 필요합니다.

2. YOLO 적용 방식

본 프로젝트에서는 YOLO를 이용해 사진 속 어류 영역을 먼저 검출합니다.

```
원본 이미지 입력
    ↓
YOLO 객체 탐지
    ↓
어류 영역 검출
    ↓
Bounding Box 기준 이미지 자르기
    ↓
배경 제거 또는 크기 조정
    ↓
어종 분류 모델 입력
    ↓
어종명 및 식용 가능 여부 출력
```

사진에 어류가 한 마리만 존재하더라도 배경, 사람 손, 낚싯대, 바닥 등이 함께 포함될 수 있으므로, YOLO로 어류 영역을 먼저 검출한 뒤 어종을 분류하는 방식으로 구성합니다.

3. 이미지 수집 및 정리

수집한 이미지를 어종별 폴더로 구분합니다.

```
raw_dataset/
├─ 가리비/
├─ 갈치/
├─ 감성돔/
├─ 고등어/
├─ 국매리복/
├─ 기름치/
├─ 별복/
└─ ...
```

각 이미지의 파일명은 다음과 같이 통일합니다.

```
감성돔_001.jpg  
감성돔_002.jpg  
감성돔_003.jpg
```

이미지 수집 시 다음 조건을 고려합니다.

- 물고기 전체 형태가 잘 보이는 사진
- 물속에서 촬영된 사진
- 실제 낚시로 포획된 사진
- 정면, 측면, 위쪽 등 다양한 촬영 각도
- 밝은 환경과 어두운 환경
- 바닥, 선박, 쿨러 등 다양한 배경
- 손, 낚싯대, 뜰채 등이 함께 등장하는 실제 환경 사진

수집된 이미지는 어종명이 정확한지 먼저 확인한 후 폴더에 저장합니다. 동일한 이미지가 반복되어 포함되지 않도록 중복 파일도 함께 검사합니다.

2. 🛠️ 체장 측정 및 규제 준수 가이드/상세 프로세스

이 기능은 컴퓨터 비전 기술과 법적 규제 데이터를 결합하여, 사용자가 올바르고 안전한 낚시 문화를 실천하도록 돕는 자동화 가이드입니다.

① 이미지 기반 데이터 전처리

- **사용자 입력:** 어종이 포함된 이미지 업로드.
- **참조 스케일 생성:** 이미지 내 물고기의 크기를 상대적으로 측정하기 위해 픽셀 단위의 데이터를 물리적 길이(cm)로 변환하는 비례식 계산 과정을 거칩니다.

② 실시간 체장 및 어종 분석

- **체장 추출:** 탐지된 물고기의 경계 상자(Bounding Box) 및 핵심 특징점 데이터를 기반으로 실제 체장을 산출합니다.

③ 법적 기준 대조 및 가이드 제공

- **데이터베이스 연동:** 식별된 어종별로 해양수산부의 법적 '금지 체장' 데이터를 MongoDB에서 실시간으로 조회합니다.
- **규제 위반 판정:** 측정된 체장이 금지 체장 기준보다 작을 경우, 즉각적인 경고 메시지를 사용자에게 송출하여 방류를 유도합니다.
- **결과 안내:** 분석된 어종의 명칭, 측정된 체장, 법적 기준 준수 여부를 포함한 최종 리포트를 출력합니다.

3. 📍 낚시 가능 구역 안내

1. 개요

본 프로젝트의 목적은 출발지와 목적지 간의 최단 경로 및 소요 시간을 산출하는 시스템을 구현하는 것임. 이를 위해 외부 상용 API 연동 방식과 자체 라우팅 엔진 구축 방식을 비교 검토하여 최적의 방안을 결정하고자 함.

2. 방안 비교

구분	방안 A: 상용 API 연동 (카카오모빌리티 등)	방안 B: 자체 라우팅 엔진 구축 (OSM + OSRM)
핵심 원리	외부 서버에 경로를 요청하여 결과값 수신	서버 내 자체 라우팅 엔진을 직접 운영
장점	개발 속도 빠름, 데이터 정확도 및 신뢰도 높음	트래픽 비용 절감, 커스터마이징 자유도 높음
단점	호출량에 따른 비용 발생, 로직 수정 불가	데이터 전처리 및 서버 운영/유지보수 필요
적합한 상황	서비스 초기, 정밀한 실시간 교통 정보 중요	대규모 트래픽 발생, 고도화된 특수 로직 필요

3. 세부 실행 계획

[방안 A] 상용 API 연동 방식

- API 키 확보:** 카카오모빌리티 등 서비스별 디벨로퍼스 계정 생성 및 API 키 발급.
- 데이터 호출:** 출발지/도착지 위경도 좌표를 API 파라미터로 전송.
- 결과 처리:** 응답받은 JSON 데이터(경로 좌표 리스트)를 지도 서비스에 렌더링.
- 기대 효과:** 최소한의 개발 리소스로 안정적인 경로 탐색 서비스 조기 런칭.

[방안 B] 자체 라우팅 엔진 구축 방식

- 데이터 확보:** OSM(OpenStreetMap) 데이터 및 공공 교통 데이터(GTFS, ITS 등) 수집.
- 엔진 구축:** Docker 기반 OSRM 혹은 GraphHopper 엔진 서버 세팅.
- 전처리 파이프라인:** 수집된 데이터를 엔진 규격에 맞게 변환 및 인덱싱.
- 가중치 로직 적용:** 실시간 교통 데이터를 연동하여 엔진 내 링크 가중치 동적 업데이트.
- 기대 효과:** 외부 의존성 제거 및 서비스 정책에 맞춘 유연한 경로 최적화 로직 적용.

4. 종합 제안 및 결론

- 제안:** **방안 A(API 연동)**를 통해 프로토타입을 신속히 구현하여 핵심 기능을 검증할 것을 권장함.
- 이유:** 개발 초기 단계에서는 라우팅 엔진의 복잡한 운영 리스크보다 사용자 경험(UX) 확보가 우선임. 추후 트래픽 증가로 인한 비용 효율성 문제나 서비스 고도화 요구가 발생할 경우 **방안 B**로 단계적 전환을 검토하는 것이 효율적임.

프로젝트 구현 소개

1. Project Structure (프로젝트 구조)

📁 디렉토리 구조 요약 본 프로젝트(**kakaoapi**)는 백엔드(**FastAPI**), 프론트엔드(**HTML/CSS/JS**), AI 모델(**YOLO**, 어종 분류) 및 데이터베이스(**MongoDB**)가 유기적으로 연결된 구조로 설계되었습니다.

```
PKNU_SUMMERBOOTCAMP_PROJECT/  
├── data/           # 서비스 내 정적/동적 데이터 보관 폴더  
└── src/
```

```

├── kakaoapi/
│   ├── __pycache__/
│   ├── data/           # API 전용 로컬 데이터셋
│   ├── database/       # 데이터베이스 쿼리 및 커넥션 세부 구현부
│   ├── routers/        # API 라우팅 모듈
│   ├── schemas/        # Pydantic 기반 데이터 검증 스키마
│   ├── services/       # 비즈니스 로직 및 에이전트 코디네이터
│   ├── static/         # 프론트엔드 정적 리소스
│   ├── templates/      # UI 프리젠테이션 템플릿
│   ├── tool/           # AI 에이전트 전용 실행 도구
│   ├── database.py      # 기본 DB 연결 및 세션 초기화 설정 파일
│   ├── main.py         # FastAPI 어플리케이션 진입점
│   └── tool1_backup.py  # 이전 도구 소스 백업본
├── yolo/               # YOLO 기반 실시간 객체 탐지 독립 모듈
├── .gitignore          # 형상 관리 제외 대상 설정 파일
├── Workflow_Diagram.png # 흐름도 이미지
├── readme.md           # 메인 프로젝트 설명서
└── readme.pdf          # 출력용 마크다운 변환 PDF 문서

```

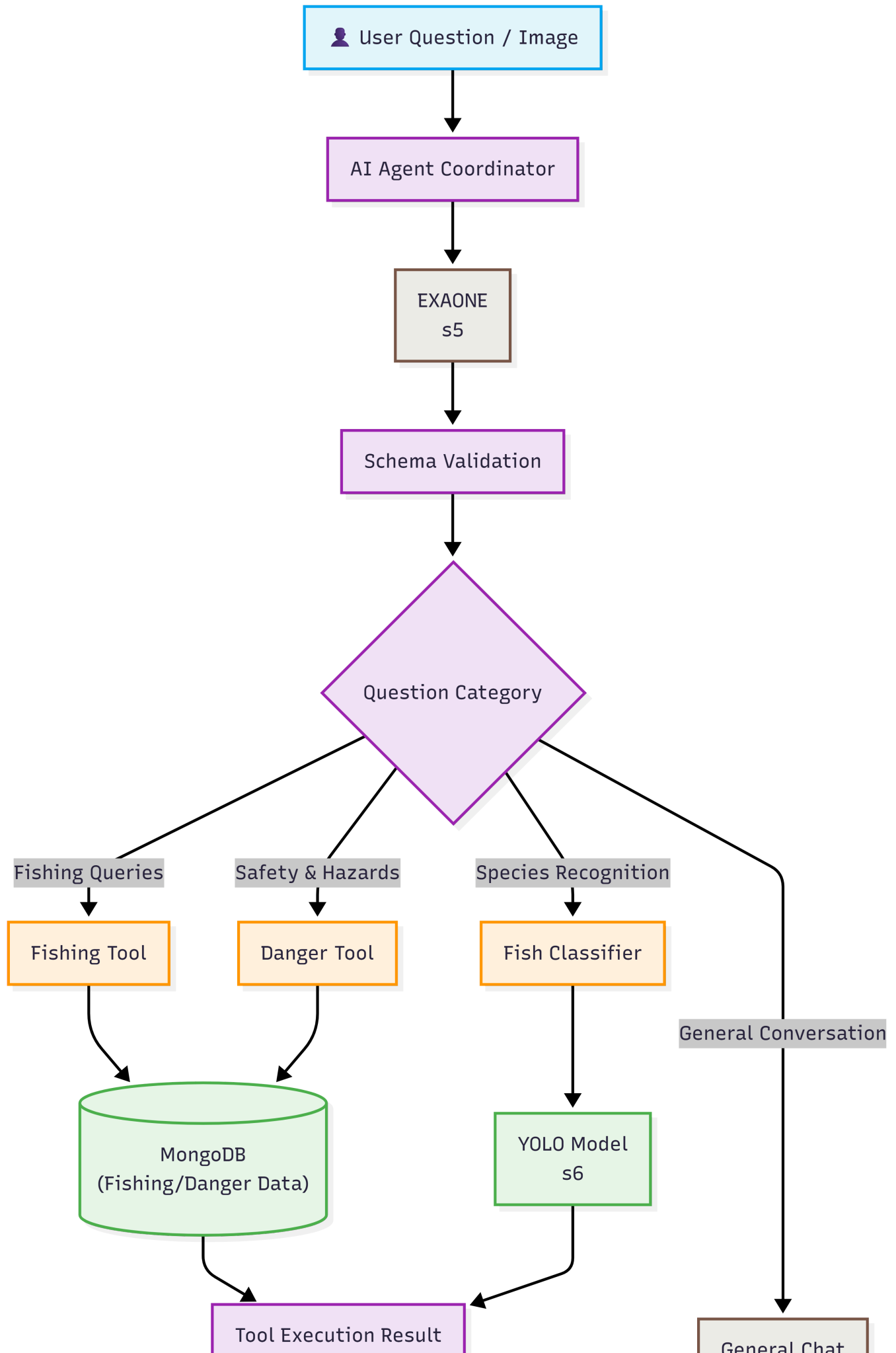
2. AI Agent Workflow (AI 에이전트 워크플로우)

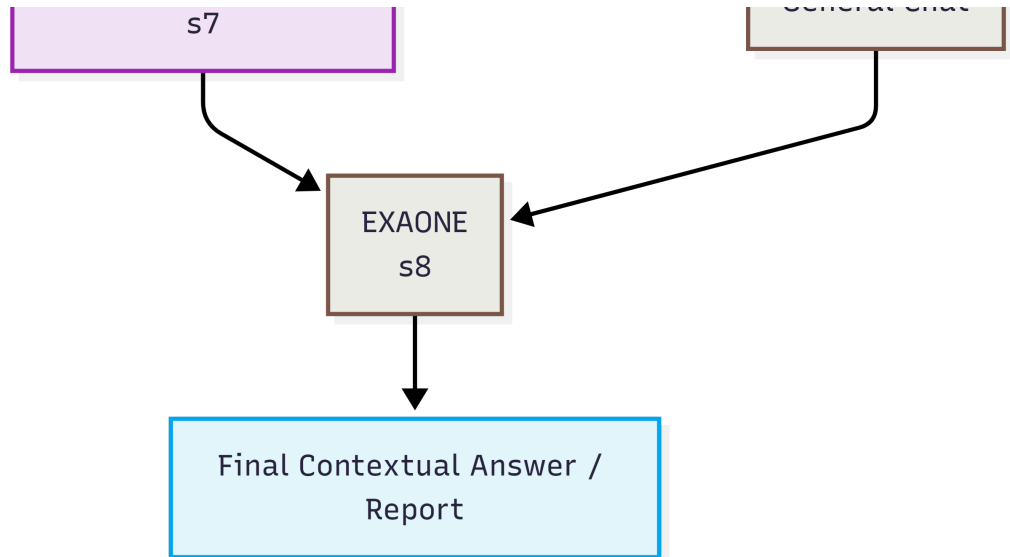
 **AI 에이전트 핵심 아키텍처** 사용자의 요청(텍스트/이미지)이 입력되면 **EXAONE LLM**이 의도를 분석하고 분류합니다. 이후 검증 단계를 거쳐 최적의 기능 도구(**Tool**)를 호출하고, 최종적으로 데이터베이스 조회 결과와 AI 모델 추론 결과를 결합하여 직관적인 자연어 답변을 도출합니다.

에이전트 작동 프로세스

- 사용자 요청 수신 (User Input):** 사용자가 채팅창을 통해 질문을 입력하거나 물고기 사진을 업로드합니다.
- 의도 분석 (Intent Classification):** **EXAONE LLM**이 사용자의 질문 맥락을 분석하여 어떤 작업(어종 분류, 낚시 규제 조회, 위험 구역 확인, 일반 대화)이 필요한지 판별합니다.
- 데이터 검증 (Schema Validation):** 정의된 Pydantic 스키마를 통해 AI 에이전트의 매개변수가 누락 없이 올바르게 매핑되었는지 검증합니다.
- 기능 도구 호출 (Tool Execution):** 판별된 카테고리에 맞는 전용 도구(**Tool Layer**)를 호출하여 실행합니다.
 - 데이터 저장소 및 모델 연동:** 각 도구는 **MongoDB**에서 관련 지리/규제 데이터를 쿼리하거나, 업로드된 이미지를 **YOLO** 모델로 전달해 실시간 객체 탐지를 수행합니다.
- 자연어 답변 생성 (NLG):** 도구 실행 결과 반환된 정형 데이터(JSON)를 **EXAONE LLM**이 다시 받아 사용자가 이해하기 쉬운 자연스러운 문장 및 가이드 리포트로 재구성하여 최종 전달합니다.

흐름도 (Mermaid Diagram)





✂ 3. 주요 기능 및 구현 상세

1 📷 이미지 기반 어종 판별 (YOLO + 어종 분류 모델)

🔍 **핵심 기능 정의** 단일 이미지 내에서도 배경, 인물 손, 낚싯대, 바닥 등 다양한 요소가 포함되기 때문에 ****객체 탐지(YOLO)****로 어류 영역을 먼저 분할(Crop)하여 불필요한 배경 노이즈를 제거한 뒤 **어종 분류 모델**의 추론 정확도를 극대화합니다.

✂ YOLO 객체 탐지 및 어종 분류 파이프라인

1. **원본 이미지 입력**: 사용자가 업로드한 낚시 결과물 사진을 시스템에 등록합니다.
2. **YOLO 객체 탐지 (Object Detection)**: 딥러닝 기반 탐지 엔진이 이미지 내에서 '어류(Fish)' 객체의 정확한 위치를 추출합니다.
3. **어류 영역 검출 및 자르기 (Localization & Crop)**: 검출된 바운딩 박스(Bounding Box) 좌표를 기준으로 어류 부분만 자릅니다.
4. **전처리 및 크기 조정 (Preprocessing)**: 잘린 이미지에서 노이즈를 제어하고, 분류 모델 규격에 맞게 리사이징을 거칩니다.
5. **어종 분류 모델 추론 (Classification)**: 정제된 이미지를 입력받아 CNN 기반 분류 엔진이 정확한 어종 명칭을 식별합니다.
6. **최종 리포트 출력**: 식별된 최종 어종명과 데이터베이스 연동 정보(식용 가능 여부 등)를 매칭하여 반환합니다.

원본 이미지 입력 → YOLO 객체 탐지 → 어류 영역 검출 → Bounding Box 이미지 자르기 → 배경 제거/조정 → 어종 분류 모델 입력 → 최종 결과 도출

✂ 데이터셋 구성 및 가이드라인

- 🌐 **식용 가능 어종 (49종)**: 어종별 최소 3장 이상의 이미지 데이터를 확보합니다.
- ⚠ **식용 불가 어종 (3종)**: **국매리복**, **기름치**, **별복**으로 정의하며, 각 어종의 구체적인 식용 불가 사유는 독립적인 JSON 포맷의 메타데이터로 연결합니다.
- 📷 **수집 검증 기준**:
 - 물속 렌더링, 실제 낚시 포획, 정면/측면/상단 다각도 촬영물 위주로 확보합니다.

- 바닥, 선박, 아이스박스, 사람 손, 뜰채 등 실제 현장 노이즈가 복합적으로 구성된 이미지를 통해 실전 정확도를 개선합니다. (※ 프로토타입 단계에서는 동작 확인을 위해 어종당 최소 3장으로 구성하나, 실서빙을 위한 모델 최적화 단계에서는 50~100장 이상의 데이터 보강이 필요합니다.)

2 🛠️ 체장 측정 및 규제 준수 가이드

🔑 **핵심 기능 정의** 컴퓨터 비전 기술과 공공 데이터베이스의 수산법 규제 데이터를 결합하여, 사용자가 올바르게 안전한 방류 문화를 실천하도록 돕는 자동화 가이드라인입니다.

🌸 체장 측정 및 비교 가이드 프로세스

- 참조 스케일 및 비례식 계산 (Pixel to Metric):** 이미지 내 어체와 대비되는 기준 객체 또는 픽셀 정밀 데이터 분석을 통해 실제 물리 길이(cm) 단위로 변환하기 위한 변환 비례 관계를 생성합니다.
- 체장 중심점 추출 (Length Measurement):** YOLO가 추출한 좌표 및 어체의 머리 끝(Head)에서 꼬리 끝(Tail)까지의 핵심 특징점을 기하학적으로 연산하여 실측 체장을 측정합니다.
- 수산자원 규제 데이터 실시간 대조 (DB Validation):** 식별된 어종명을 인덱싱하여 MongoDB에 적재된 해양수산부 수산자원관리법상의 '포획 금지 체장' 데이터를 즉시 쿼리합니다.
- 금지 체장 위반 판정 및 경고 (Compliance Check):** 분석된 실측 체장이 규정 크기 미만일 경우, 법적 처벌 위험성 안내 및 방류를 유도하는 팝업 UI를 노출합니다.
- 피드백 리포트 생성:** 측정 길이, 기준 체장 정보, 법적 준수 적격 여부가 한눈에 포함된 카드 형태의 요약 리포트를 출력합니다.

3 📍 낚시 가능 구역 및 이동 경로 최적화 안내

🔑 **핵심 기능 정의** 사용자의 출발지부터 대상 낚시 포인트까지 최적의 동선 및 소요 시간을 제안하고, 맵 레이어 오버레이를 통해 해당 좌표가 낚시 허용 구역인지 여부를 시각적으로 실시간 필터링합니다.

🌸 경로 탐색 인프라 설계 비교 분석

구분	🛠️ 방안 A: 상용 API 연동 (카카오모빌리티 등)	⚙️ 방안 B: 자체 라우팅 엔진 구축 (OSM + OSRM)
핵심 기술	외부 기업의 검증된 API 엔드포인트 동기 호출	오픈스트리트맵(OSM) 지도 로드 및 자체 호스팅 OSRM 구동
장점	빠른 기능 배포(Time-to-Market), 최신 실시간 교통 데이터 및 정밀 경로 탐색 신뢰성 확보	트래픽 폭증 시 누적 API 호출 비용 전면 제거, 전용 라우팅 세부 가중치(비포장도로 우회 등) 맞춤 개발 가능
단점	플랫폼 호출 스케일 확장에 따른 과금 리스크, 외부 인프라 장애 종속 및 기능 개별 튜닝 불가	대용량 맵 데이터 초기 인덱싱/전처리 리소스 부담, 전용 인프라 관리 부담 및 유지 보수 어려움
적합한 환경	서비스 기획 초기 검증 단계, 소규모 사용자 검증 시	특수 차량/선박 전용 길찾기 필요 시, 트래픽이 상시 대량화된 안정화 단계

🌸 종합 제안 및 결론

🚀 **의사 결정 가이드** 개발 초기에는 인프라 최적화보다 비즈니스 가치 검증과 사용자 피드백 반영이 최우선입니다. 따라서 **[방안 A] 카카오모빌리티 API 연동 방식**을 기반으로 맵 인터페이스 레이아웃을 신속히 설계하고,

추후 대량의 안정적 사용 트래픽이 유지되는 시점에 비용 합리화를 위해 [방안 B] OSRM 엔진의 자체 이관을 진행하는 점진적 아키텍처 스케일링이 가장 효율적입니다.

4 🗺️ 낚시 지수 기반 AI 챗봇

☀️ **핵심 기능 정의** 단순 텍스트 질의응답을 넘어 기상 및 바다 조석 상태를 반영하여, 해당 지역의 조파를 예측하고 맞춤형 낚시 전술 조언을 제공하는 Context-Aware 인공지능 비서 서비스입니다.

🌸 주요 챗봇 연동 설계

- 실시간 기후·환경 메트릭스 수집:** 기상청 및 국립해양조사원 API를 백그라운드로 연결하여 풍속, 파고, 수온, 물때(조석 예보) 등의 최신 포인트 데이터를 획득합니다.
- 지능형 적합 지수 산출 (Activity Index):** 수집된 다차원 환경 데이터를 결합하여 오늘 목적지의 '종합 낚시 적합 지수'를 내부 모델을 통해 정량적으로 평가하고 시각적 인디케이터로 제공합니다.
- 컨텍스트 맞춤형 피드백 제공 (Contextual NLG):** EXAONE LLM에 기후 정량 메트릭을 시스템 프롬프트 컨텍스트로 전달하여, 사용자의 질의에 따라 가장 전문적이고 직관적인 맞춤형 팁(낚시 기법, 주의사항 등)을 대화로 풀어 제공합니다.

🧩 4. System Components (시스템 컴포넌트)

프론트엔드부터 서비스 에이전트, 하이퍼-로컬 데이터 스토어까지의 기술적 역할 정의 및 책임 매핑 정보입니다.

계층 (Layer)	핵심 컴포넌트 (Components)	기술적 역할 정의 및 설명
🌐 Front-End	location.html location.css location.js	사용자의 브라우저 GPS 좌표 추출, 카카오맵 API 연동을 통한 지리 경계 가시화 및 낚시 안전 레이어 시각화, 모바일 친화적 위젯 렌더링.
🚀 FastAPI	main.py chat.py (routers)	비동기 입출력 구조의 API 엔드포인트 설계. 모듈러 구조로 라우터를 라우팅하여 백엔드 컴포넌트 간 트래픽 흐름 조율 및 성능 극대화.
🤖 AI Agent	agent.py (services)	세션 관리 및 이전 대화 기록을 프롬프트 컨텍스트로 결합하는 메모리 관리자. 정형 스키마 검증을 기반으로 한 LLM 미들웨어 역할.
🧠 EXAONE	LLM Engine	에이전트의 중심 두뇌. 사용자의 질의로부터 필요한 변수 (Argument)를 파싱 및 함수 매핑 처리하고, JSON 분석 원시 정보를 친절한 완성형 텍스트로 보강하여 변환.
🔧 Tool Layer	fishing_tool.py danger_tool.py fish_classifier.py	LLM이 필요로 하는 행동을 독자적으로 분리하여 구현한 독립 실행 함수군. 원시 파일 전처리, 서드파티 예측 추론 및 DB 액세스 격리 수행.
🗄️ DB & ML Inference	MongoDB (지리/포획 정보) YOLO (컴퓨터 비전)	공공 수산 데이터 및 안전 구역 지리 포인트 상시 적재/관리. 업로드 이미지 내 어류 객체의 실시간 좌표 및 클래스 확률 정밀 감지 엔진 구동.